Homework 2

Agentic Systems with Hardware

Due date: 12.10.2026

How to submit

Write one PDF. For Task 1 it describes the system, shows the two runs, and gives the GitHub link to the Task 1 code. For Task 2 it is a short report: the experiment table, the focusing images, and the comparison with the lecture, with the GitHub link to the Task 2 code.

Push the code to a GitHub repository. The repository may be the one you used for Homework 1, Task 6, with this work added. Put the screen recording in that repository as well, or attach it to the email together with the PDF.

Send one email to `aramsargsyan@ysu.am` with that PDF attached.

Task 1. Human-in-the-loop

Homework 1, Task 6, is a LangGraph system of four roles: an orchestrator, a scientist, a general agent, and an executor with a coding tool. The user speaks only to the orchestrator, and the orchestrator chooses the next role from the dialogue. That system runs until it finishes or until it has made eight decisions. This task keeps that system and adds a stop, so that some decisions are made by a person before the graph continues.

Problem.

Start from your Homework 1, Task 6 solution. Keep the four roles, the shared dialogue, and the rule that the orchestrator's choice of the next role is a model call.

When the graph stops. The orchestrator can now choose a fifth destination, `human`, beside `scientist`, `general`, `executor`, and `finish`. It chooses `human` in two situations.

- An **ambiguous query**. The latest request has more than one reading, and those readings would send the work to different roles or would change the number that gets reported. The graph asks which reading to follow.
- A **hard decision**. The next step would run the coding tool, overwrite a result that is already on disk, or end the dialogue and report a measurement. The graph states that step and asks whether to proceed.

The choice is a model call that reads the dialogue and returns the destination, the question to ask, and a one-sentence reason. A list of keywords, or a rule that always stops on the same role, does not count.

How the stop works. The stop is a LangGraph interrupt. Compiling the graph with a checkpointer, and calling it on a thread, is part of the system. When the orchestrator chooses `human`, the graph pauses and surfaces the question. It stays paused until you resume that same thread with your reply. The reply is appended to the dialogue, with the role name `human`, and the following turn can read it. On an ambiguous query, the next decision follows the reading you gave. On a hard decision, the step runs after you accept it. If you refuse, the turn returns to the orchestrator and the step does not run.

What to show. Run the extended system twice.

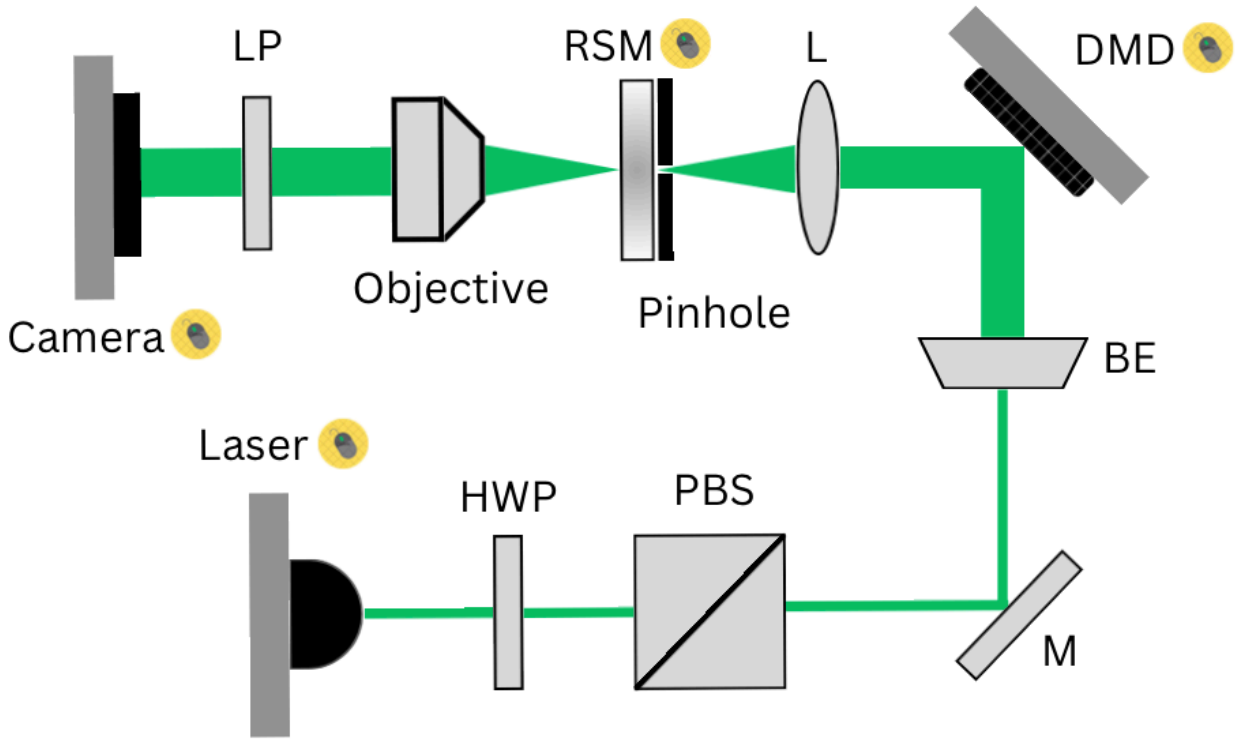
(a) Give it an ambiguous request about the Silverbox work from Homework 1. In the PDF, include the question the graph posed, the reply you typed, and the next message in the dialogue. 5 points

(b) Give it a request that leads to a hard decision, and refuse once before you accept. In the PDF, include both questions, both replies, and the message that shows the refused step did not run. 5 points

Push the code for this task to a GitHub repository and put the link in the PDF.

Task 2. Focusing through a simulated bench

The figure is the laboratory path: a laser, a digital micromirror device (DMD), a scattering medium, and a camera, with the optomechanics that sit between them.



This task simulates the part of that path that forms the image. Light goes from the laser to the DMD, through the scattering medium, and onto the camera. The DMD changes only the amplitude of E_{in}. With T the transmission matrix of that path, the camera records

$$E_{\text{out}} = |TE_{\text{in}}|^2 .$$

Add noise to that measurement, so a second shot of the same pattern is not a copy of the first.

The goal is a focus: a bright spot in a chosen region of the camera, standing out from the speckle around it. Follow this algorithm to obtain the focus.

- Divide the DMD into macropixels and start from a random pattern.
- After each transmission, read the intensity in the focus region and in the proximal background.
- Split the background into several regions, and take the enhancement as the focus against that background.
- Flip some macropixels, see how the enhancement changed, and repeat.

Problem.

Build a tool that runs this search on the simulated bench and returns the enhancement and the camera images. Add that tool to the agent from Homework 1, Task 6. The executor is the role that calls it.

Ask the agent to focus the beam. From that request on, the agent works on its own: it chooses the settings, including the macropixel size, the region sizes, and the number of iterations, it calls the tool, and it keeps the enhancement the tool returns. You do not type those settings in during the run.

In the report, give a table of the experiments the agent ran. Each row has the settings for that run and the enhancement it obtained. Include the camera images, and compare the focus you got with the focusing result from the lecture. **Screen record a video** of the agent making a decision, calling the tool, and running an experiment. Add the video to the GitHub repository, or send it by email with the PDF. 14 points